Empirical Study of Sorting Algorithms

Moumit Bera

1 Introduction

Sorting is a fundamental problem in computer science with applications in data processing, searching, and algorithm design. While asymptotic complexity provides theoretical bounds on performance, empirical analysis reveals how algorithms behave under different input distributions.

This project investigates the theoretical and experimental behavior of sorting algorithms.

2 Insertion Sort

2.1 Theoretical Analysis

Best Case:

$$T(n) = \Theta(n)$$

Worst Case:

$$T(n) = \Theta(n^2)$$

Average Case:

$$T(n) = \Theta(n^2)$$

2.2 Experimental Setup

Metrics recorded:

- Comparisons
- Shifts
- Execution Time (ms)

Array sizes:

$$n = 1000, 2000, 5000, 10000, 20000, 50000$$

2.3 Detailed Results Table

Input Type	N	Comparisons	Shifts	Run Time (ms)
Sorted	1000	999	0	0.016
	2000	1999	0	0.015
	5000	4999	0	0.016
	10000	9999	0	0.074
	20000	19999	0	0.150
	50000	49999	0	0.374
Reverse Sorted	1000	499500	499500	1.304
	2000	1999000	1999000	4.337
	5000	12497500	12497500	18.097
	10000	49995000	49995000	69.263
	20000	199990000	199990000	272.022
	50000	1249975000	1249975000	1700.455
Random	1000	248288	247295	1.040
	2000	1004094	1002101	2.727
	5000	6242496	6237505	10.000
	10000	25077297	25067305	34.422
	20000	100175744	100155755	136.546
	50000	624949001	624899012	849.583
Duplicate Heavy	1000	237236	236239	0.992
	2000	952758	950761	2.561
	5000	5942234	5937238	9.565
	10000	23774823	23764827	33.194
	20000	95014971	94994975	129.383
	50000	593828027	593778030	806.148
Nearly Sorted	1000	34132	33133	0.173
	2000	122705	120706	0.602
	5000	764749	759750	2.324
	10000	3109850	3099851	5.880
	20000	12459258	12439259	18.295
	50000	77414619	77364620	105.793

2.4 Graphical Analysis

To clearly visualize growth patterns across different input types, a logarithmic scale was used on the Y-axis.

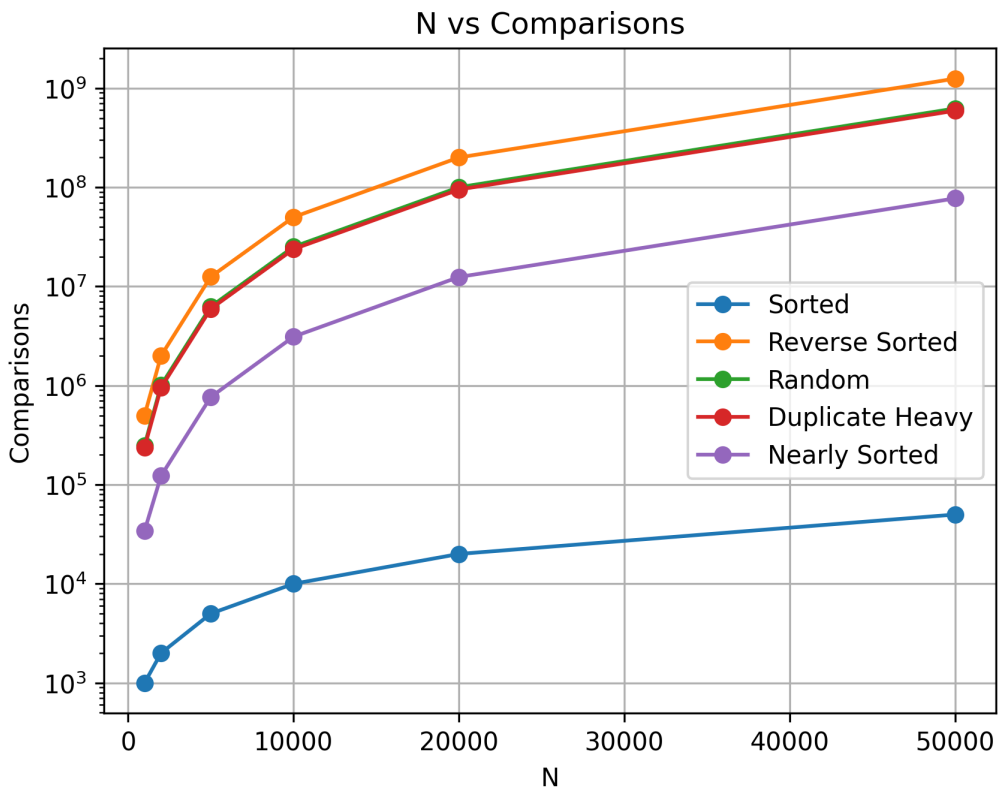

Figure 1: Growth of Comparisons with respect to n (Log Scale)

Figure 2: Growth of Shifts with respect to n (Log Scale)

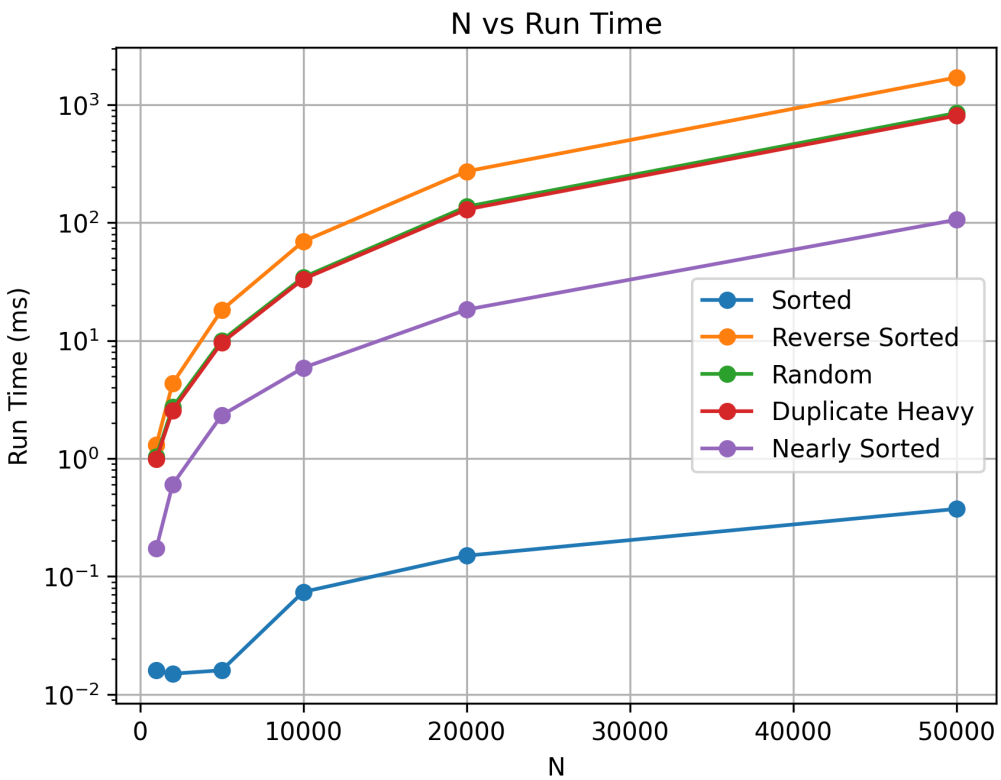

Figure 3: Growth of Execution Time with respect to n (Log Scale)

2.5 Discussion

- Sorted arrays exhibited linear growth.
- Reverse sorted arrays exhibited quadratic growth.
- Random and duplicate-heavy inputs followed quadratic trends.
- Nearly sorted arrays demonstrated near-linear behavior.

The runtime correlates directly with the inversion count.

3 Merge Sort

3.1 Theoretical Analysis

Best Case:

$$T(n) = \Theta(n \log n)$$

Worst Case:

$$T(n) = \Theta(n \log n)$$

Average Case:

$$T(n) = \Theta(n \log n)$$

Auxiliary Space:

$$S(n) = \Theta(n)$$

3.2 Experimental Setup

Metrics recorded:

- Comparisons
- Writes
- Execution Time (ms)

Array sizes:

$$n = 1000, 2000, 5000, 10000, 20000, 50000$$

3.3 Detailed Results Table

Input Type	N	Comparisons	Writes	Run Time (ms)
Sorted	1000	5044	19952	0.162
	2000	11088	43904	0.221
	5000	32004	123616	0.841
	10000	69008	267232	1.534
	20000	148016	574464	2.416
	50000	401952	1568928	5.021
Reverse Sorted	1000	4932	19952	0.157
	2000	10864	43904	0.349
	5000	29804	123616	0.823
	10000	64608	267232	0.878
	20000	139216	574464	2.402
	50000	382512	1568928	5.073
Random	1000	8702	19952	0.162
	2000	19401	43904	0.297
	5000	55231	123616	1.227
	10000	120452	267232	2.190
	20000	260866	574464	2.950
	50000	718186	1568928	7.713
Duplicate Heavy	1000	8616	19952	0.214
	2000	19196	43904	0.389
	5000	54597	123616	1.087
	10000	118924	267232	1.881
	20000	257324	574464	3.163
	50000	707493	1568928	5.637
Nearly Sorted	1000	7654	19952	0.089
	2000	17252	43904	0.371
	5000	50630	123616	0.645
	10000	111601	267232	1.626
	20000	243420	574464	2.533
	50000	673777	1568928	5.087

3.4 Graphical Analysis

To clearly visualize asymptotic growth trends across different input types, a logarithmic scale was used on both axes.

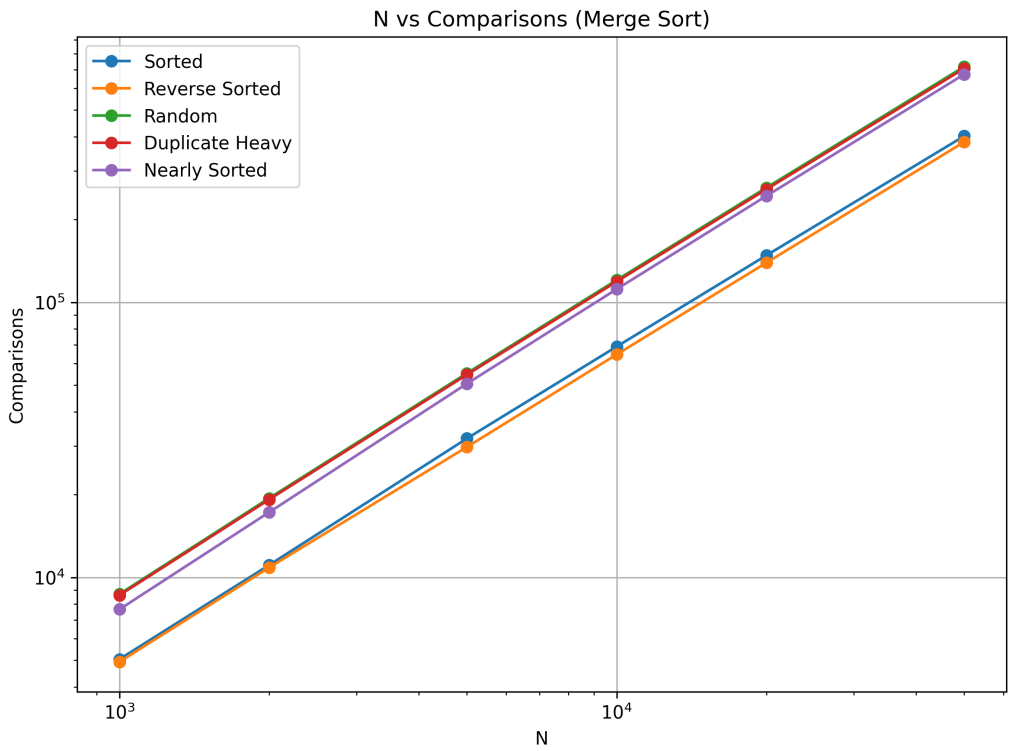

Figure 4: Growth of Comparisons with respect to n (Log-Log Scale)

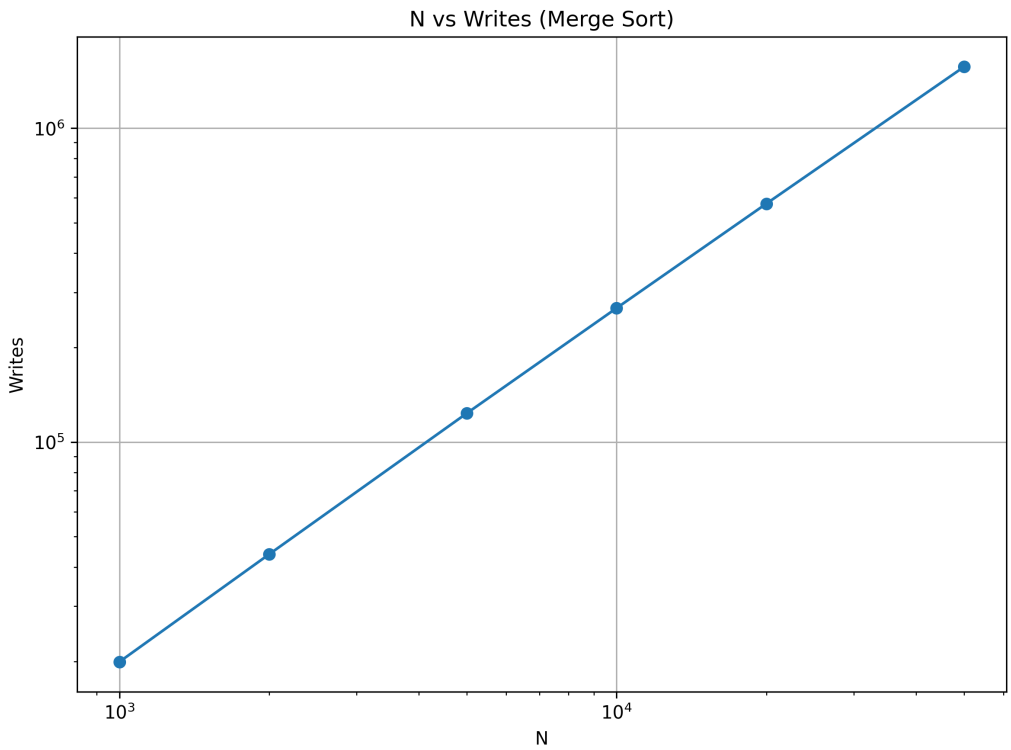

Figure 5: Growth of Writes with respect to n (Log-Log Scale)

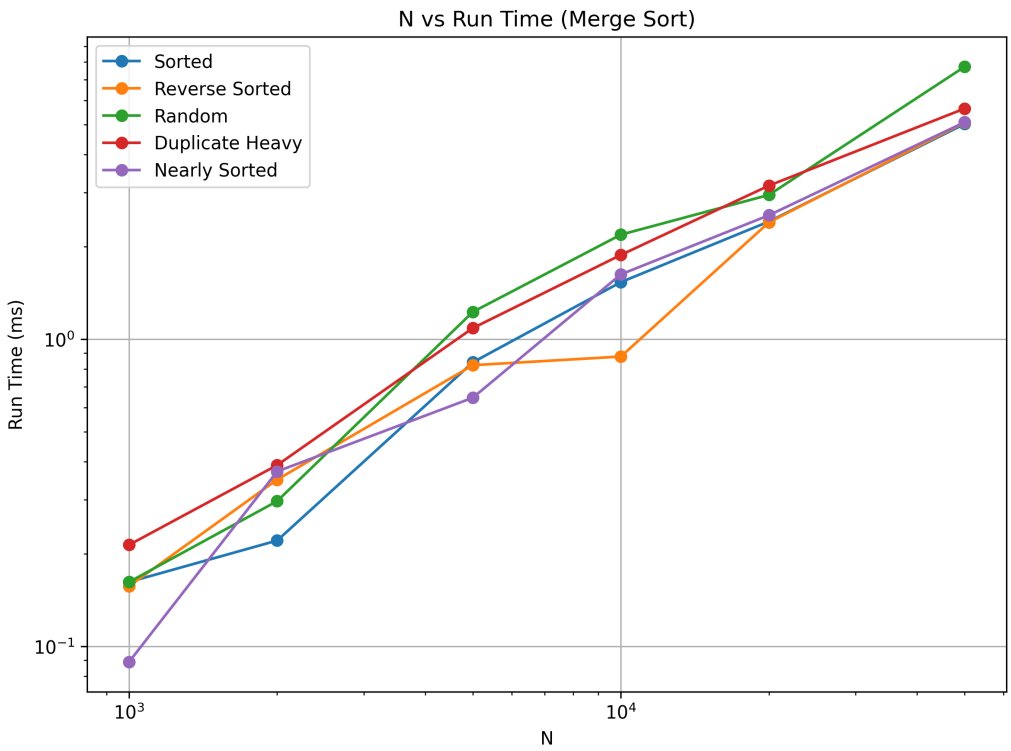

Figure 6: Growth of Execution Time with respect to n (Log-Log Scale)

3.5 Discussion

- All input distributions exhibited $\Theta(n \log n)$ growth.
- Comparisons varied slightly depending on input order.
- Writes remained nearly constant across all cases.
- Unlike Insertion Sort, performance was largely independent of inversion count.

Empirical observations closely matched theoretical expectations, confirming the divide-and-conquer structure of Merge Sort.

4 Quick Sort

4.1 Theoretical Analysis

Worst Case (Unbalanced Partition):

$$T(n) = \Theta(n^2)$$

Average Case (Balanced Partition in Expectation):

$$T(n) = \Theta(n \log n)$$

Best Case (Perfectly Balanced Partition):

$$T(n) = \Theta(n \log n)$$

Auxiliary Space (Recursion Stack):

$$S(n) = \Theta(\log n) \text{ (average)}, \quad \Theta(n) \text{ (worst case)}$$

4.2 Experimental Setup

Quick Sort was implemented using the Lomuto partition scheme with multiple pivot selection strategies:

- Last Element Pivot
- Random Pivot
- Median-of-Three Pivot

The following metrics were recorded:

- Comparisons
- Swaps
- Execution Time (ms)

Array sizes:

$$n = 1000, 2000, 5000, 10000, 20000, 50000$$

For each input size, the following distributions were tested:

- Sorted
- Reverse Sorted
- Random
- Duplicate-Heavy
- Nearly Sorted

Each experiment was averaged over multiple trials.

4.3 Pivot Strategy I: Last Element Pivot

4.3.1 Detailed Results Table

Input Type	N	Comparisons	Writes	Run Time (ms)
Sorted	1000	499500	500499	2.612
	2000	1999000	2000999	6.116
	5000	12497500	12502499	31.379
	10000	49995000	50004999	123.657
	20000	199990000	200009999	487.679
	50000	1249975000	1250024999	3042.939
Reverse Sorted	1000	499500	250499	2.365
	2000	1999000	1000999	6.002
	5000	12497500	6252499	27.811
	10000	49995000	25004999	111.774
	20000	199990000	100009999	434.212
	50000	1249975000	625024999	2736.875
Random	1000	11093	6602	0.113
	2000	24606	13972	0.198
	5000	70462	40539	0.857
	10000	154872	86768	1.042
	20000	335646	186663	2.132
	50000	939010	519042	5.480
Duplicate Heavy	1000	29860	28294	0.329
	2000	110470	106745	0.798
	5000	650148	642472	2.523
	10000	2552856	2537276	7.282
	20000	10108304	10079031	26.917
	50000	62747071	62672281	159.106
Nearly Sorted	1000	35060	18768	0.278
	2000	87712	54296	0.339
	5000	232756	147878	1.384
	10000	528747	322017	2.260
	20000	1141526	684700	2.916
	50000	3275758	1868302	9.345

4.3.2 Graphical Analysis

To visualize growth patterns across different pivot strategies and input types, a logarithmic scale was used on both axes.

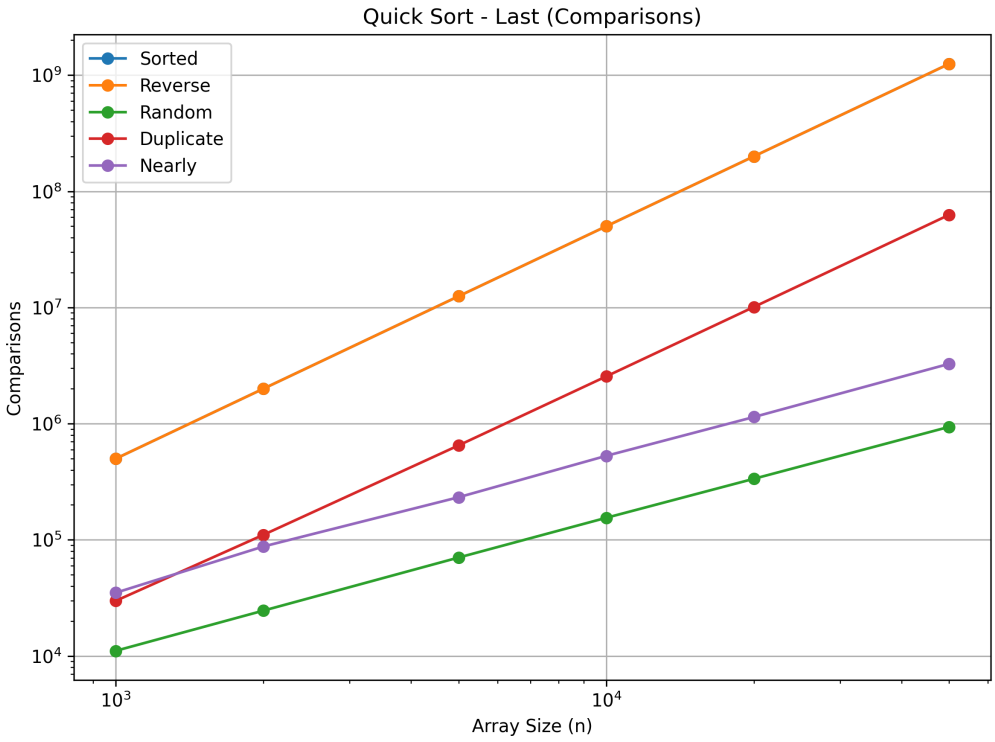

Figure 7: Growth of Comparisons with respect to n (Log-Log Scale)

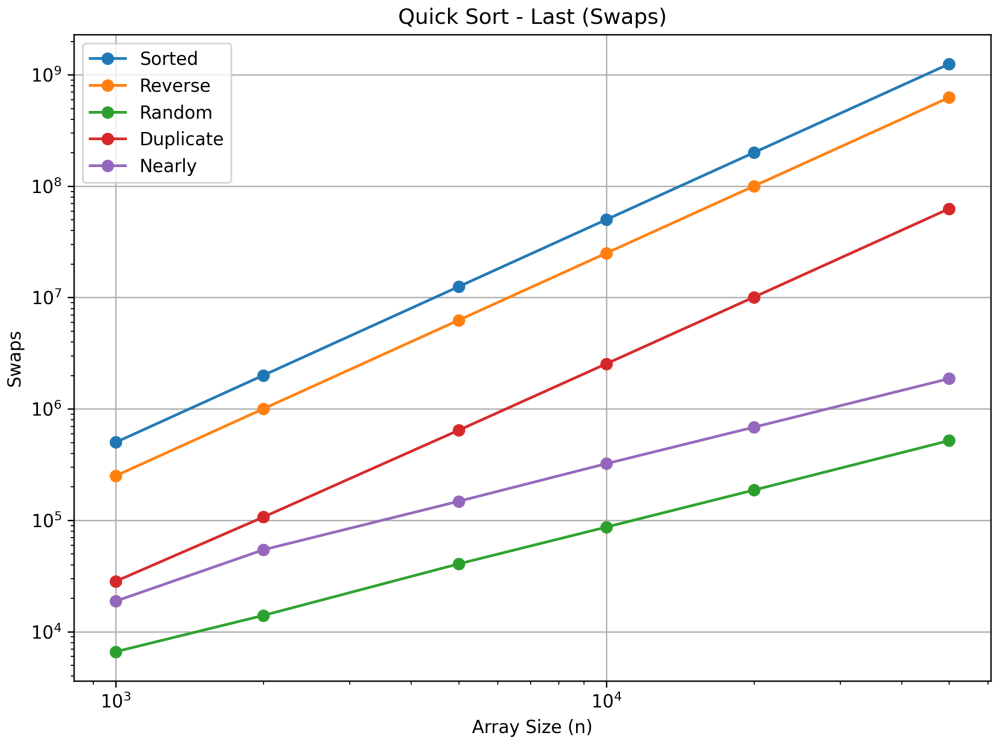

Figure 8: Growth of Swaps with respect to n (Log-Log Scale)

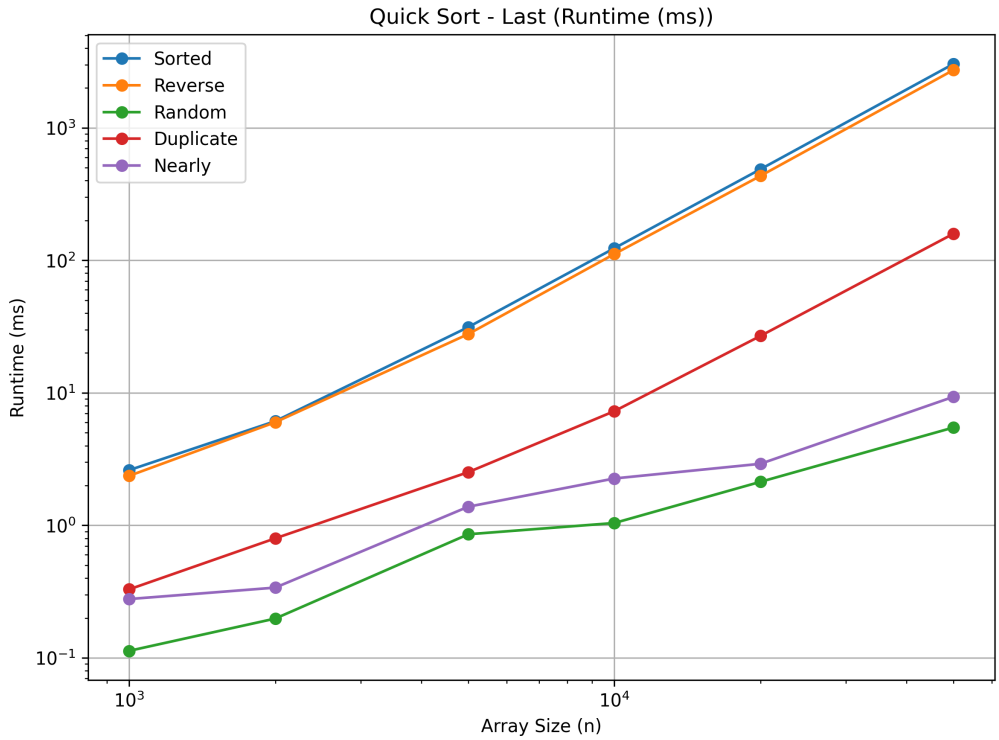

Figure 9: Growth of Execution Time with respect to n (Log-Log Scale)

4.3.3 Discussion

- When using the last-element pivot, sorted and reverse-sorted inputs exhibited $\Theta(n^2)$ growth due to highly unbalanced partitions.
- Random pivot selection significantly reduced the likelihood of worst-case behavior, yielding empirical $\Theta(n \log n)$ growth across most input types.
- Median-of-three pivot selection further improved partition balance and reduced constant factors.
- Unlike Merge Sort, Quick Sort performance is highly sensitive to pivot selection strategy.

The experimental results highlight the probabilistic nature of Quick Sort. While worst-case quadratic behavior remains theoretically possible, appropriate pivot selection strategies substantially mitigate its practical occurrence.

4.4 Pivot Strategy II: Random Pivot

To mitigate the deterministic worst-case behavior observed with fixed pivot selection, a randomized pivot strategy was implemented. At each recursive partition step, a pivot element was selected uniformly at random from the current subarray and swapped with the last element before applying the Lomuto partition scheme.

This modification preserves the theoretical worst-case complexity of $\Theta(n^2)$, since extremely unbalanced partitions remain possible. However, the probability of repeatedly selecting the smallest or largest element as pivot across recursive levels becomes negligibly small. Consequently, the expected running time becomes $\Theta(n \log n)$ across arbitrary input distributions.

4.4.1 Detailed Results Table

Input Type	N	Comparisons	Writes	Run Time (ms)
Sorted	1000	11005	6915	0.125
	2000	24798	15260	0.253
	5000	71099	42925	0.654
	10000	152395	90054	0.972
	20000	340851	199013	1.916
	50000	949808	558023	4.324
Reverse Sorted	1000	10965	7006	0.129
	2000	24880	14724	0.268
	5000	70841	41206	0.691
	10000	154477	90018	1.058
	20000	338505	194880	2.254
	50000	931313	530666	3.726
Random	1000	11067	7060	0.080
	2000	24965	15700	0.225
	5000	70637	43158	0.638
	10000	155077	92753	1.321
	20000	335635	199184	2.942
	50000	944695	551320	5.335
Duplicate Heavy	1000	30179	29653	0.176
	2000	109882	109006	0.977
	5000	649842	647505	3.147
	10000	2552566	2546928	8.248
	20000	10105662	10091429	27.216
	50000	62757000	62737197	158.370
Nearly Sorted	1000	29928	29147	0.294
	2000	25127	15291	0.289
	5000	71853	42085	0.347
	10000	152985	90537	1.424
	20000	338681	196585	2.370
	50000	929037	522281	4.204

4.4.2 Graphical Analysis

To analyze the asymptotic growth behavior, a logarithmic scale was used on both axes. In contrast to the fixed pivot strategy, sorted and reverse-sorted inputs no longer exhibited quadratic growth. Instead, all input distributions demonstrated empirical behavior consistent with $\Theta(n \log n)$ complexity.

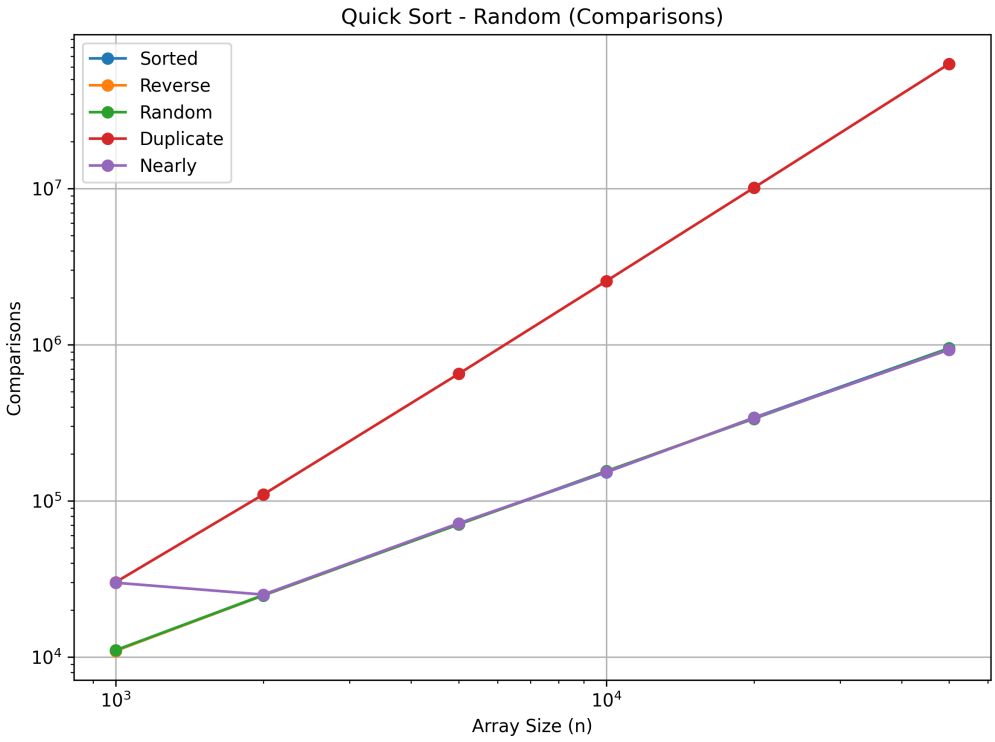

Figure 10: Growth of Comparisons with respect to n (Log-Log Scale)

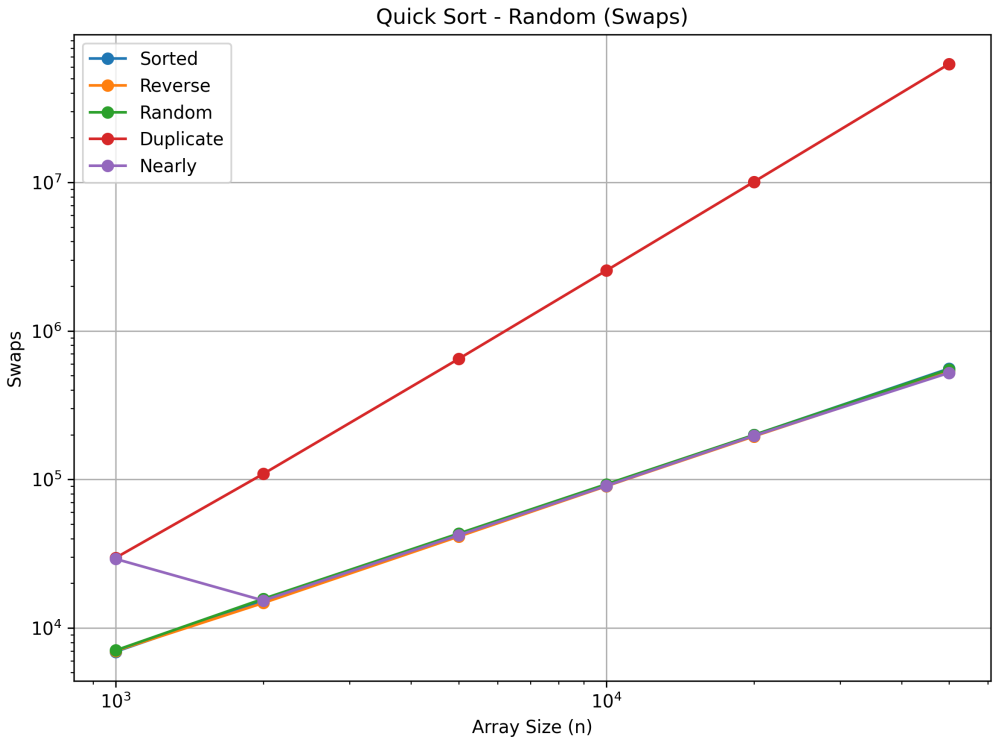

Figure 11: Growth of Swaps with respect to n (Log-Log Scale)

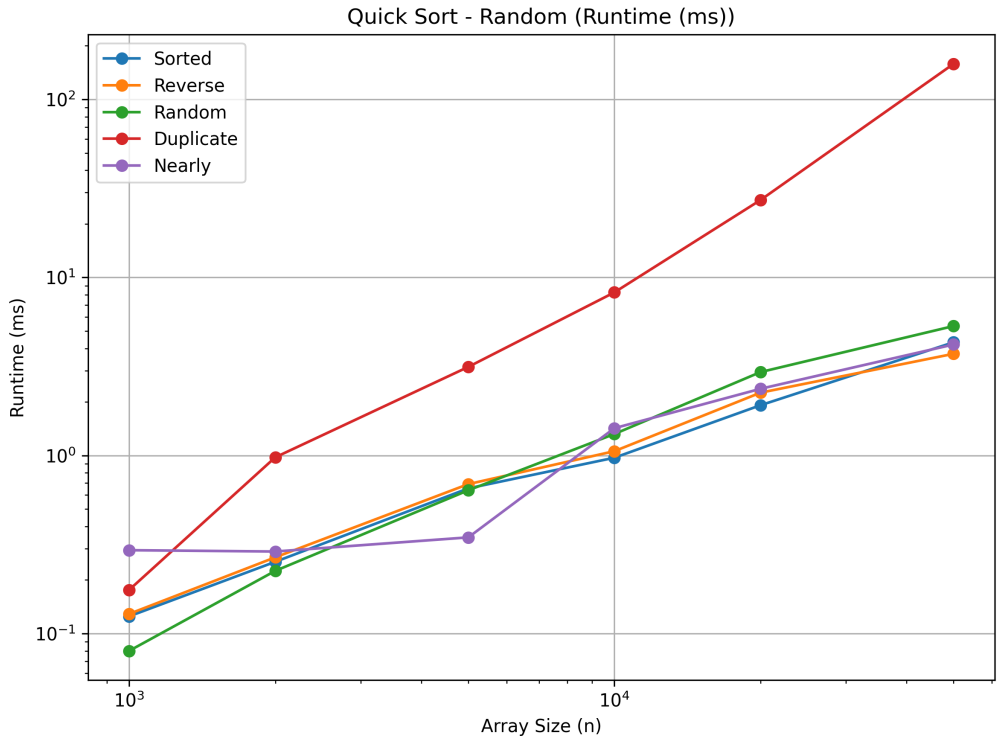

Figure 12: Growth of Execution Time with respect to n (Log-Log Scale)

4.4.3 Discussion

- Sorted and reverse-sorted inputs no longer triggered worst-case quadratic behavior.
- Empirical growth across all input types closely followed $\Theta(n \log n)$.
- Runtime variance across different distributions was significantly reduced compared to the fixed pivot strategy.
- Although worst-case $\Theta(n^2)$ behavior remains theoretically possible, its occurrence probability is extremely small in practice.

The randomized pivot strategy demonstrates how probabilistic techniques can stabilize algorithmic performance without altering asymptotic worst-case guarantees.

4.5 Pivot Strategy III: Median-of-Three Pivot

To further improve partition balance and reduce sensitivity to partially ordered inputs, a median-of-three pivot selection strategy was implemented. In this approach, the pivot is chosen as the median of the first, middle, and last elements of the current subarray. The selected median element is then swapped with the last position before applying the Lomuto partition scheme.

This heuristic aims to approximate a "good" pivot by avoiding extreme values, particularly in sorted or nearly sorted arrays. Although the theoretical worst-case time complexity remains $\Theta(n^2)$, the likelihood of highly unbalanced partitions is significantly reduced compared to the fixed pivot strategy.

Empirically, this strategy often achieves more stable partition sizes and reduced constant factors relative to both fixed and randomized pivot approaches.

4.5.1 Detailed Results Table

Input Type	N	Comparisons	Writes	Run Time (ms)
Sorted	1000	9520	4960	0.038
	2000	21033	10916	0.171
	5000	60678	30713	0.372
	10000	131343	66421	0.465
	20000	282672	142837	1.583
	50000	782782	398052	3.198
Reverse Sorted	1000	15849	9583	0.111
	2000	36177	21655	0.299
	5000	106182	63403	0.654
	10000	234923	139308	0.858
	20000	518623	306792	2.505
	50000	1433133	837620	3.895
Random	1000	11359	6730	0.145
	2000	25132	14563	0.335
	5000	70679	40710	0.894
	10000	152315	85888	1.436
	20000	329516	184615	2.865
	50000	898486	505524	4.807
Duplicate Heavy	1000	32293	29055	0.174
	2000	114788	108243	0.937
	5000	661839	645353	2.989
	10000	2576089	2543247	8.087
	20000	10150915	10083102	27.452
	50000	62872424	62704376	158.825
Nearly Sorted	1000	14592	5147	0.079
	2000	33803	11482	0.270
	5000	91932	33384	0.431
	10000	211518	70675	1.403
	20000	460362	150710	2.420
	50000	1276004	414648	4.817

4.5.2 Graphical Analysis

To evaluate asymptotic growth trends, a logarithmic scale was applied on both axes. Compared to the fixed pivot strategy, the median-of-three approach demonstrated substantially improved robustness on sorted and nearly sorted inputs. Growth patterns across all input distributions closely followed $\Theta(n \log n)$ behavior.

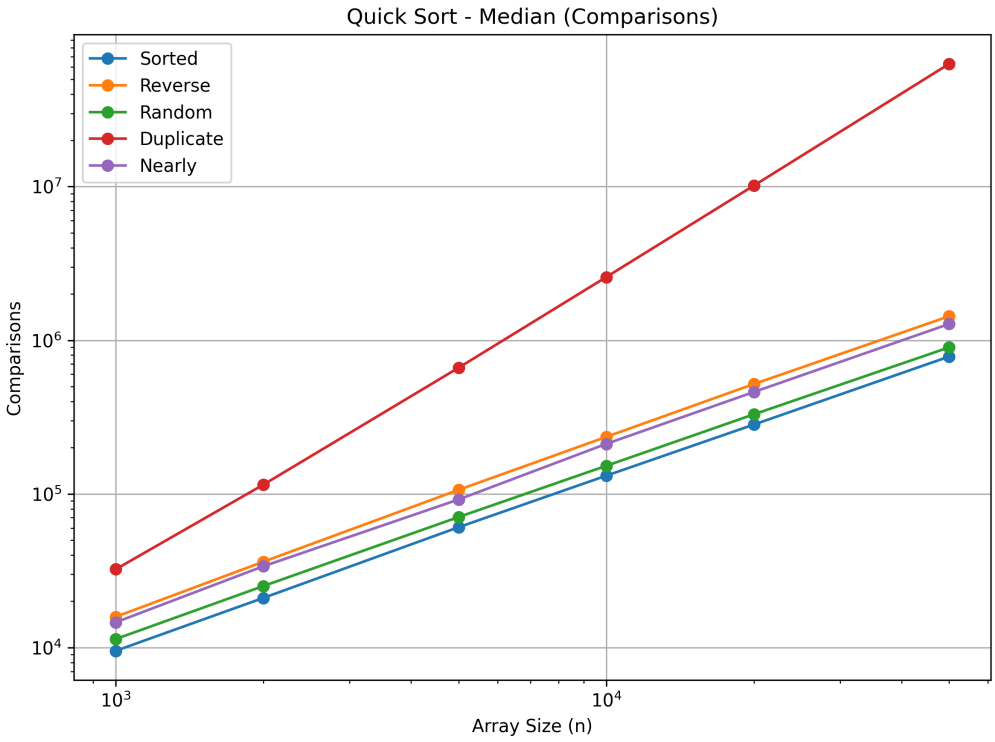

Figure 13: Growth of Comparisons with respect to n (Log-Log Scale)

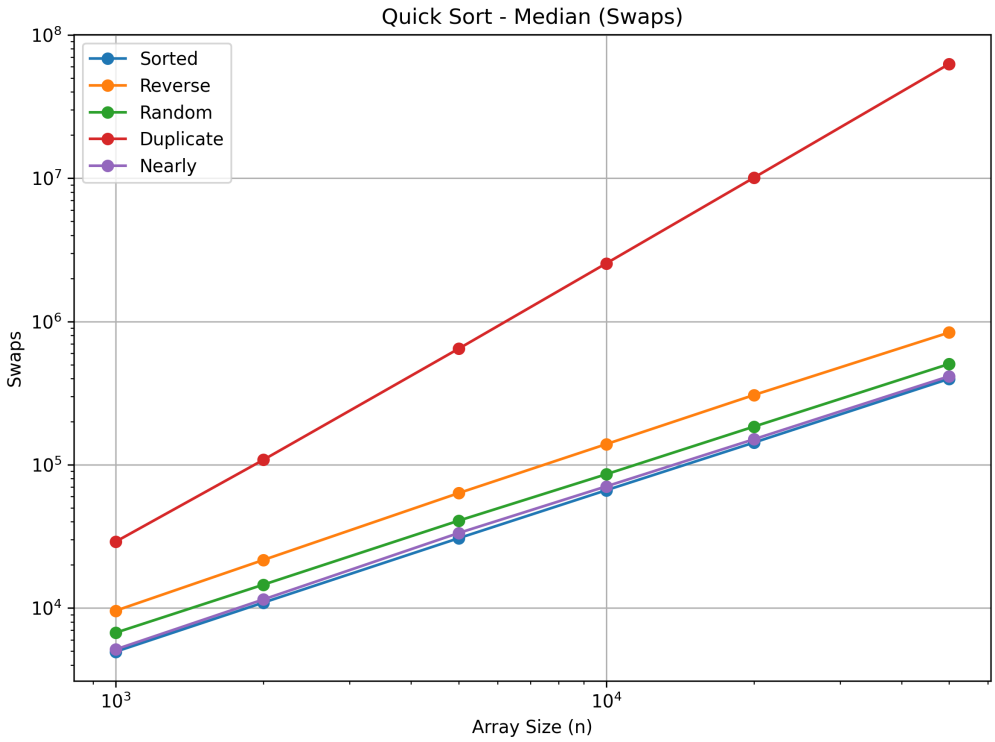

Figure 14: Growth of Swaps with respect to n (Log-Log Scale)

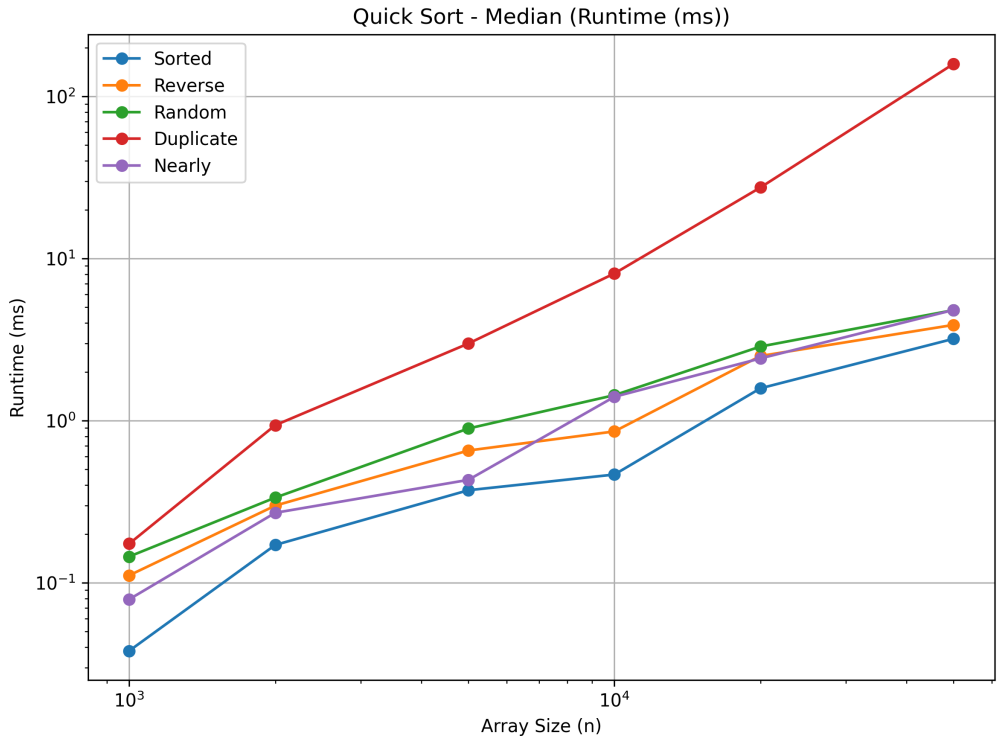

Figure 15: Growth of Execution Time with respect to n (Log-Log Scale)

4.5.3 Discussion

- Sorted and nearly sorted inputs exhibited stable $\Theta(n \log n)$ growth, avoiding the catastrophic quadratic behavior seen in the fixed pivot strategy.
- Compared to the randomized pivot strategy, the median-of-three method showed reduced runtime variance across trials.
- Constant factors were generally lower due to improved partition balance.
- Duplicate-heavy inputs still revealed limitations of the Lomuto scheme, though performance remained superior to the fixed pivot case.

The median-of-three strategy illustrates how simple deterministic heuristics can substantially improve practical performance without altering theoretical complexity bounds.

4.6 Pivot Strategy IV: Median-of-Three with 3-Way Partition

To address the performance degradation observed on duplicate-heavy inputs, the Quick Sort implementation was extended using a three-way partitioning scheme (Dutch National Flag algorithm) combined with median-of-three pivot selection.

Unlike the traditional Lomuto partition scheme, which divides the array into two regions ($\leq$ pivot and $>$ pivot), the three-way partitioning strategy divides the array into three regions:

$$< \text{ pivot} \quad | \quad = \text{ pivot} \quad | \quad > \text{ pivot}$$

This modification ensures that elements equal to the pivot are grouped together in a single pass and excluded from further recursive calls. As a result, the algorithm avoids repeatedly re-partitioning duplicate elements.

Although the theoretical worst-case time complexity remains $\Theta(n^2)$, the three-way partition strategy significantly improves performance on inputs containing many repeated values. In the extreme case where all elements are identical, the algorithm completes in linear time $\Theta(n)$.

4.6.1 Detailed Results Table

Input Type	N	Comparisons	Writes	Run Time (ms)
Sorted	1000	21665	19930	0.168
	2000	57882	54427	0.370
	5000	215416	206937	1.588
	10000	588191	571450	3.120
	20000	1618052	1585041	5.247
	50000	6216650	6133788	18.236
Reverse Sorted	1000	17401	15651	0.150
	2000	45284	41811	0.457
	5000	164131	155575	0.677
	10000	440765	423844	1.915
	20000	1196249	1162817	5.085
	50000	4537390	4454857	14.248
Random	1000	10369	8705	0.163
	2000	23190	19869	0.342
	5000	65506	57232	0.916
	10000	144977	128402	1.596
	20000	313527	280328	2.999
	50000	848999	766018	5.281
Duplicate Heavy	1000	4212	3166	0.062
	2000	8117	6070	0.060
	5000	20539	15491	0.287
	10000	41634	31587	0.560
	20000	80272	60224	0.832
	50000	205335	155287	1.797
Nearly Sorted	1000	12178	10510	0.122
	2000	27444	24087	0.292
	5000	77100	68644	0.810
	10000	163694	146736	1.340
	20000	351928	351928	1.569
	50000	968784	883585	4.925

4.6.2 Graphical Analysis

To evaluate growth behavior across different input distributions, a logarithmic scale was used on both axes.

Figure 16: Growth of Comparisons with respect to n (Log-Log Scale)

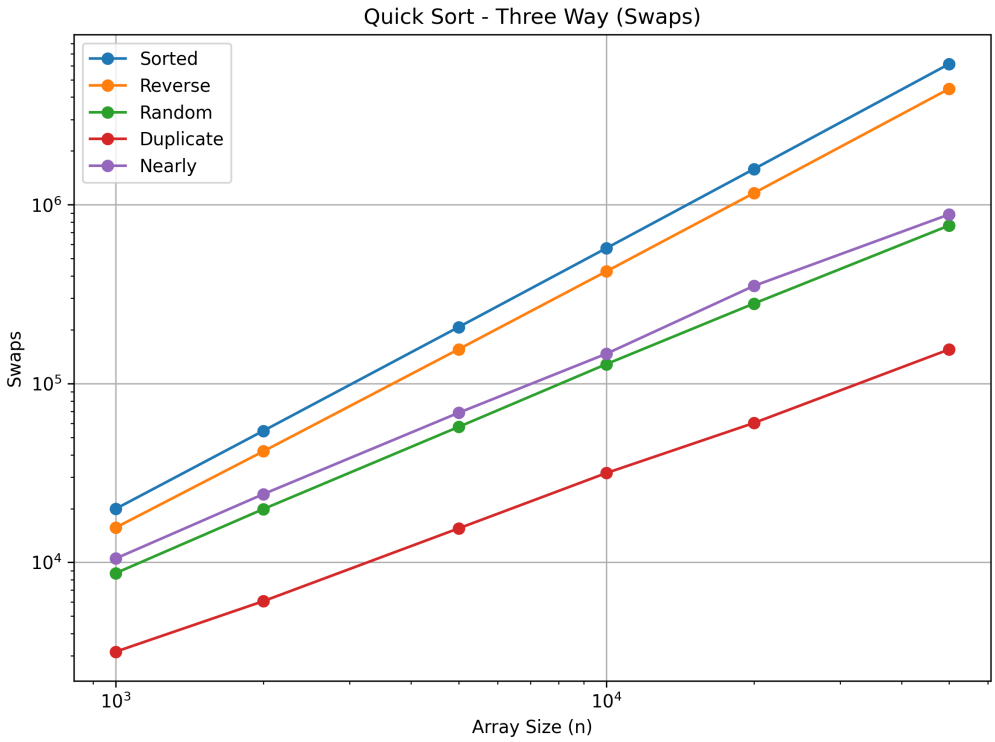

Figure 17: Growth of Swaps with respect to n (Log-Log Scale)

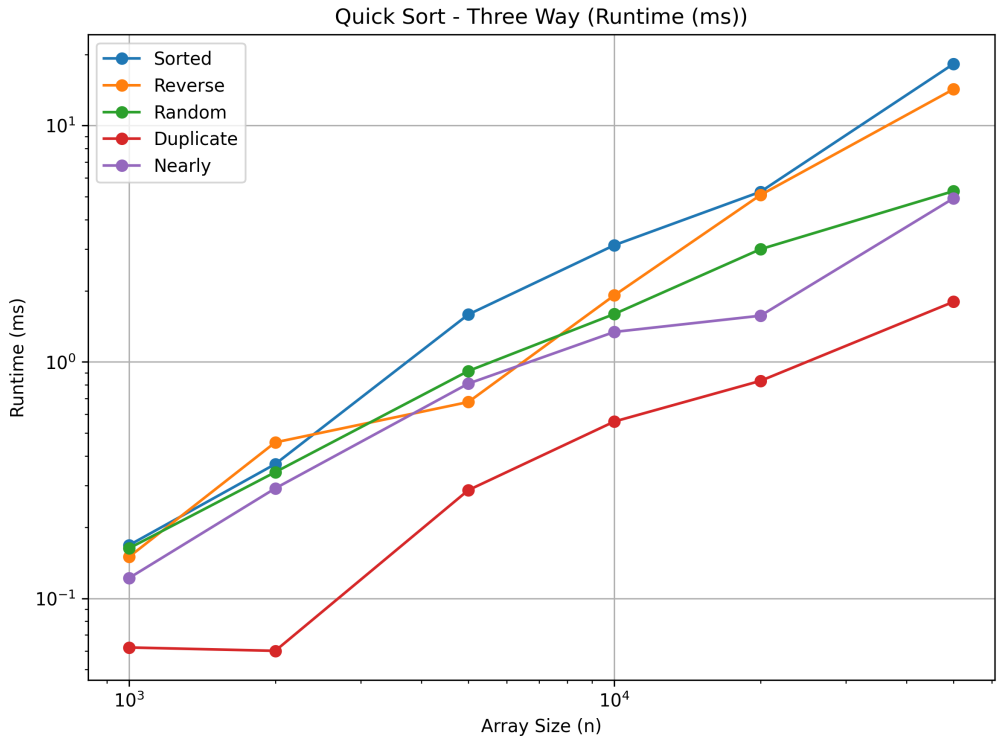

Figure 18: Growth of Execution Time with respect to n (Log-Log Scale)

4.6.3 Discussion

- Duplicate-heavy inputs exhibited a dramatic reduction in comparisons and runtime compared to Lomuto-based strategies.
- In cases where many elements were equal, performance approached linear behavior.
- For random and nearly sorted inputs, empirical growth remained consistent with $\Theta(n \log n)$.
- The three-way partition strategy reduces unnecessary recursive calls and lowers recursion depth when duplicates are present.

The results demonstrate that partition strategy plays a crucial role in practical Quick Sort performance. While pivot selection influences partition balance, the choice of partitioning scheme determines robustness under non-uniform input distributions.

4.6.4 Cross-Strategy Comparison

To further highlight the impact of partition strategy, a direct comparison of runtime across all pivot strategies was conducted under duplicate-heavy input. The dramatic reduction in execution time for the three-way partition variant is clearly visible in the log-log plot below.

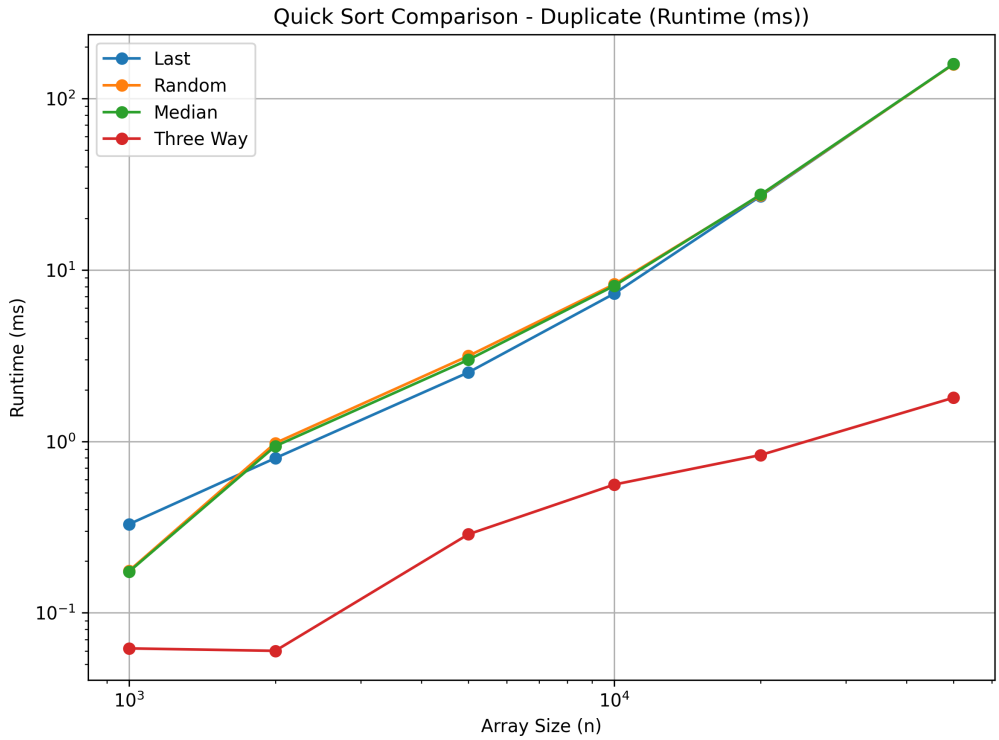

Figure 19: Runtime Comparison Across Pivot Strategies (Duplicate-Heavy Input, Log-Log Scale)

5 Overall Comparative Analysis

Across all experiments, clear distinctions emerged between quadratic-time and divide-and-conquer sorting algorithms.

Insertion Sort vs Merge Sort

Insertion Sort demonstrated excellent performance for:

- Small input sizes
- Nearly sorted inputs

However, its quadratic growth made it impractical for large disordered arrays. In contrast, Merge Sort maintained stable $\Theta(n \log n)$ growth across all input distributions, confirming its theoretical robustness.

Quick Sort Behavior

Quick Sort exhibited highly input-dependent behavior under fixed pivot selection. The last-element pivot strategy resulted in catastrophic $\Theta(n^2)$ behavior for sorted and reverse-sorted inputs.

Introducing randomized pivot selection stabilized performance and produced empirical $\Theta(n \log n)$ growth across distributions.

The median-of-three pivot strategy further improved partition balance, reducing constant factors and runtime variance.

Finally, the three-way partition strategy demonstrated dramatic improvements for duplicate-heavy inputs, reducing comparisons and recursion depth. In such cases, empirical behavior approached linear time.

Key Observations

- Pivot selection significantly influences partition balance.
- Partition strategy determines robustness under non-uniform distributions.
- Merge Sort offers predictable performance but requires additional space.
- Quick Sort offers superior practical speed when implemented with appropriate heuristics.

6 Heap Sort

6.1 Theoretical Analysis

Heap Sort is a comparison-based sorting algorithm that utilizes a binary heap data structure. The algorithm consists of two primary phases: heap construction and repeated extraction of the maximum element.

Best Case:

$$T(n) = \Theta(n \log n)$$

Average Case:

$$T(n) = \Theta(n \log n)$$

Worst Case:

$$T(n) = \Theta(n \log n)$$

Auxiliary Space:

$$S(n) = \Theta(1)$$

Heap Sort is an in-place algorithm and does not require additional memory proportional to input size, unlike Merge Sort.

6.2 Experimental Setup

Heap Sort was implemented using a max-heap representation stored in a 0-indexed array. The heap construction phase was performed bottom-up, followed by repeated extraction of the root element and heap restoration using the heapify procedure.

The following metrics were recorded:

- Comparisons
- Swaps
- Execution Time (ms)

Array sizes:

$$n = 1000, 2000, 5000, 10000, 20000, 50000$$

For each input size, the following distributions were tested:

- Sorted
- Reverse Sorted
- Random
- Duplicate-Heavy
- Nearly Sorted

Each experiment was averaged over multiple trials.

6.3 Detailed Results Table

Input Type	N	Comparisons	Swaps	Run Time (ms)
Sorted	1000	17583	9708	0.097
	2000	39159	21300	0.441
	5000	112126	60932	1.241
	10000	244460	131956	2.035
	20000	529074	282878	3.748
	50000	1455438	773304	6.988
Reverse Sorted	1000	15965	8316	0.111
	2000	35964	18708	0.334
	5000	103227	53436	1.233
	10000	226682	116696	2.208
	20000	493307	254334	3.514
	50000	1366047	698892	6.431
Random	1000	16834	9061	0.176
	2000	37713	20156	0.320
	5000	107679	57110	0.952
	10000	235392	124211	2.429
	20000	510769	268426	3.170
	50000	1409872	737469	8.692
Duplicate Heavy	1000	16447	8724	0.219
	2000	36610	19299	0.551
	5000	104328	54640	1.320
	10000	227573	118732	2.209
	20000	492690	256236	3.849
	50000	1357730	703605	7.617
Nearly Sorted	1000	17484	9647	0.289
	2000	39065	21348	0.292
	5000	111705	60388	1.289
	10000	243495	130821	1.622
	20000	527166	281754	3.853
	50000	1451360	771193	7.403

6.4 Graphical Analysis

To analyze growth behavior, a logarithmic scale was applied to both axes. Heap Sort exhibited consistent $\Theta(n \log n)$ growth across all input distributions.

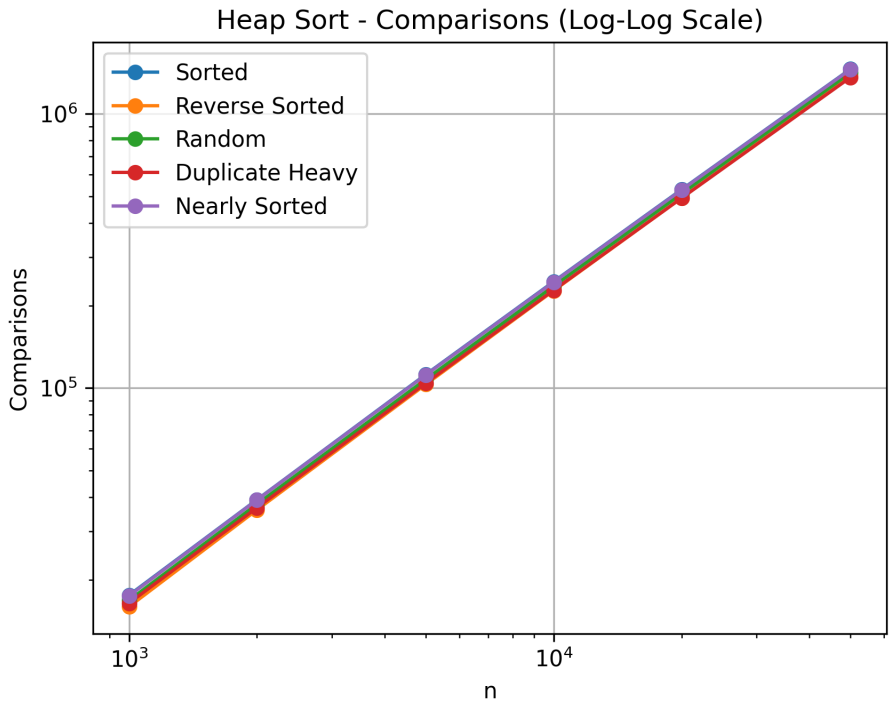

Figure 20: Growth of Comparisons with respect to n (Log-Log Scale)

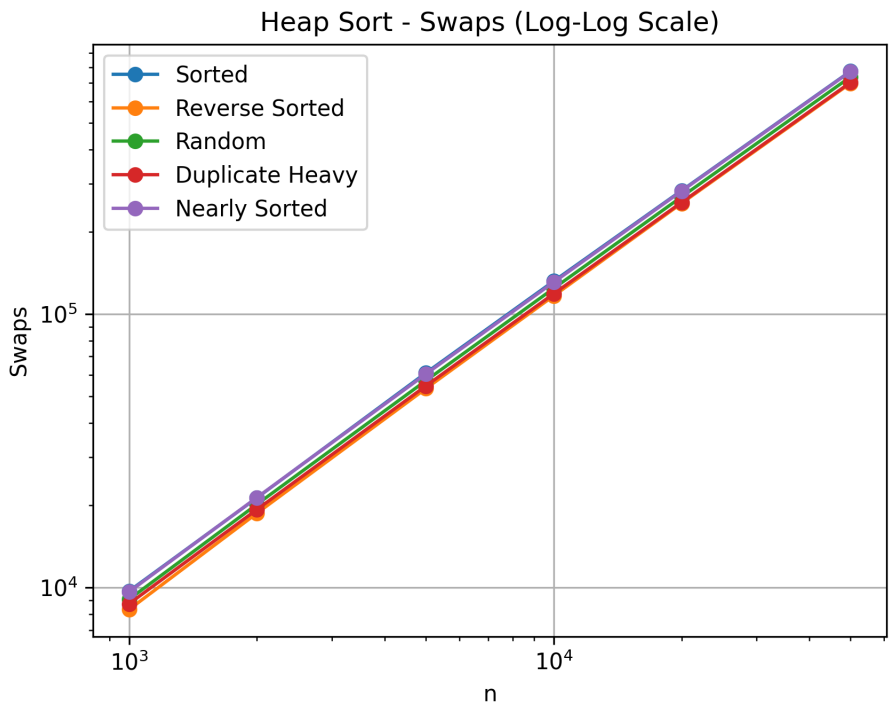

Figure 21: Growth of Swaps with respect to n (Log-Log Scale)

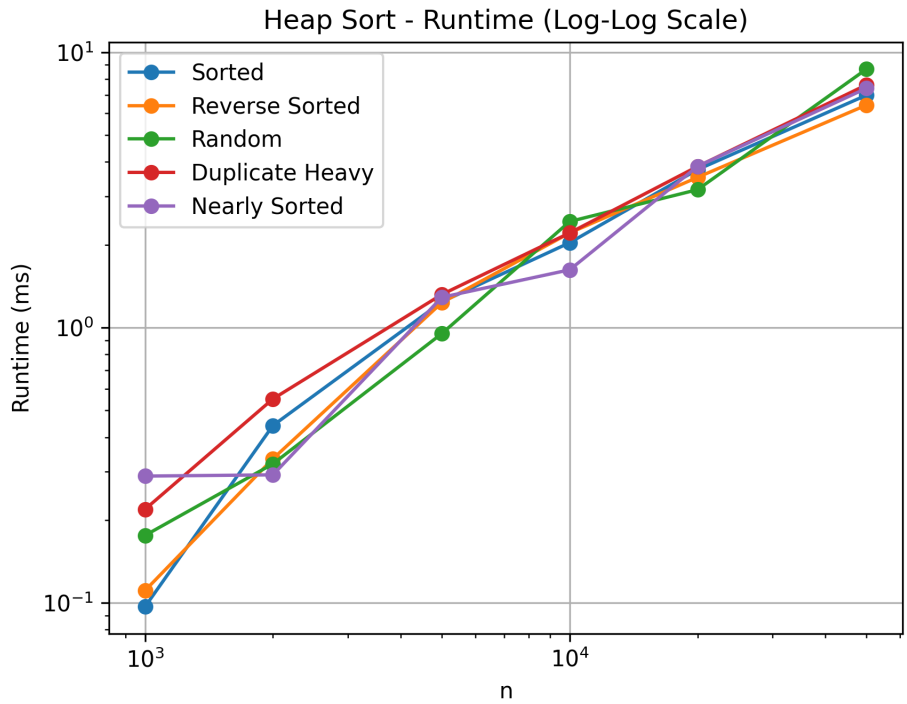

Figure 22: Growth of Execution Time with respect to n (Log-Log Scale)

6.5 Discussion

- Heap Sort demonstrated consistent $\Theta(n \log n)$ behavior across all input distributions.
- Unlike Quick Sort, performance was largely insensitive to input order.
- Duplicate-heavy inputs did not cause performance degradation.
- The number of swaps was generally higher compared to Merge Sort.
- Although asymptotically optimal, Heap Sort exhibited larger constant factors compared to optimized Quick Sort variants.

Heap Sort provides strong worst-case guarantees and in-place operation, making it robust and memory-efficient. However, empirical results indicate that optimized Quick Sort implementations typically outperform Heap Sort in practical runtime due to better cache locality and lower constant factors.

6.6 Cross-Algorithm Comparison

When compared against Insertion Sort, Merge Sort, and the various Quick Sort pivot strategies:

- Heap Sort consistently outperformed Insertion Sort for large n.
- Merge Sort demonstrated slightly more stable write behavior.
- Median-of-three Quick Sort generally achieved lower runtime constants.
- Three-way Quick Sort significantly outperformed Heap Sort on duplicate-heavy inputs.

These results highlight the trade-offs between theoretical guarantees, memory usage, and practical performance in sorting algorithm design.

7 Conclusion

This empirical investigation validates classical theoretical complexity results while revealing important practical insights into sorting algorithm behavior under varied input distributions.

Insertion Sort demonstrated strong performance for small and nearly sorted inputs, exhibiting linear behavior when inversion count was minimal. However, its quadratic growth rendered it unsuitable for large, highly disordered datasets.

Merge Sort consistently achieved stable $\Theta(n \log n)$ performance across all input types. Its behavior was largely insensitive to initial ordering, confirming the predictability of its divide-and-conquer structure. The primary trade-off observed was its $\Theta(n)$ auxiliary space requirement.

Quick Sort exhibited the greatest sensitivity to implementation choices. When using a fixed pivot (last element), sorted and reverse-sorted inputs resulted in catastrophic $\Theta(n^2)$ behavior. However, randomized pivot selection significantly reduced worst-case occurrences in practice, yielding empirical $\Theta(n \log n)$ growth. The median-of-three heuristic further improved partition balance and reduced constant factors. Most notably, combining median-of-three pivot selection with three-way partitioning dramatically improved performance on duplicate-heavy inputs, approaching linear behavior in extreme cases.

Heap Sort provided deterministic $\Theta(n \log n)$ performance independent of input distribution. While it avoided the worst-case vulnerabilities of naive Quick Sort implementations and required only constant auxiliary space, its practical runtime constants were generally higher than optimized Quick Sort variants.

Overall, the study demonstrates that asymptotic complexity alone does not fully characterize real-world algorithm performance. Implementation strategies—particularly pivot selection and partitioning schemes—substantially influence constant factors, stability, and robustness under non-uniform data distributions. Careful algorithm engineering can transform theoretically vulnerable algorithms into highly efficient practical solutions.